

6.5830/1 Lecture 12: Transactions (ctd.)

3/19/2026

Tianyu Li

litianyu@mit.edu

Administrivia

- Lab 2 due today
- Lab 3 out today
 - You should be ready to tackle parts 1 and 2
 - Logging discussed after spring break
- Quiz 1 grades will be released soon
 - Will adjust based on distribution
- Sign up for project meeting before you leave!

Recap: 2PL

- Acquire locks only in growth phase, release only in shrink phase
 - There are serializable schedules that would not be allowed by 2PL (view vs. conflict serializable)
- May still have “dirty reads”, cascading aborts
 - Solution: Strong Strict 2PL (aka Rigorous 2PL)

You will implement this in GoDB

PSA: Locks vs. Latches

	<i>Locks</i>	<i>Latches</i>
Separate...	Transactions	Workers (threads, processes)
Protect...	Database Contents	In-Memory Data Structures
During...	Entire Transactions	Critical Sections
Modes...	Shared, Exclusive, Update , Intention	Read, Write
Deadlock	Detection & Resolution	Avoidance
...by...	Waits-for, Timeout, Aborts	Coding Discipline
Kept in...	Lock Manager	Protected Data Structure e.g. page frame

Recap: Deadlocks

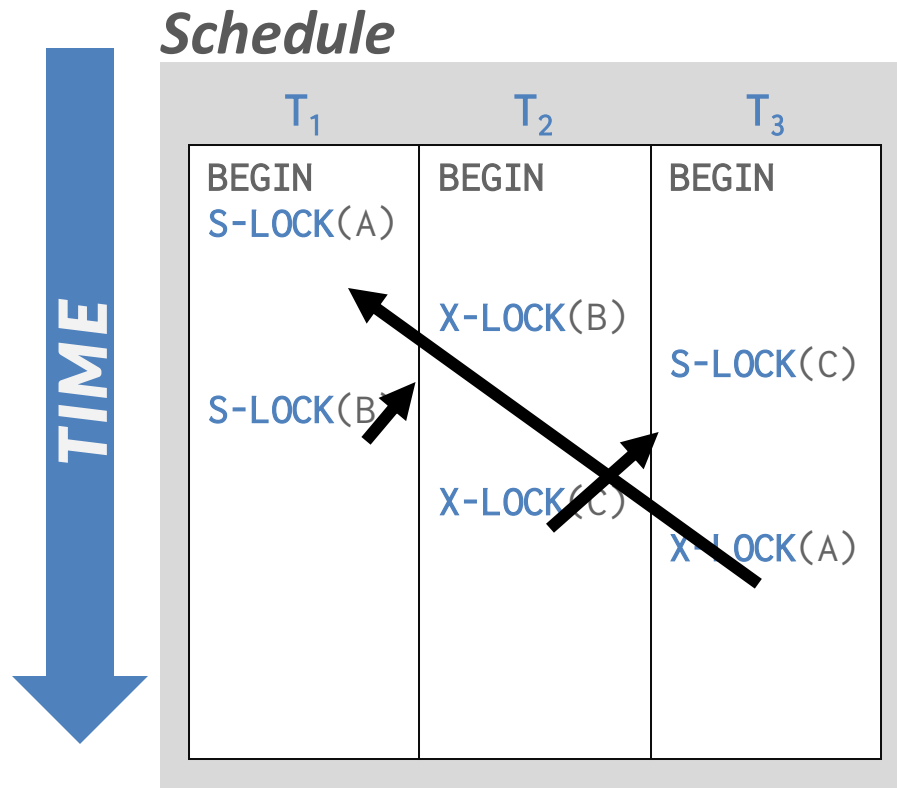
- A deadlock is a cycle of transactions waiting for locks to be released by each other.
- Two ways of dealing with deadlocks:
 - Approach #1: Deadlock Detection
 - Approach #2: Deadlock Prevention

Your design choice in GoDB

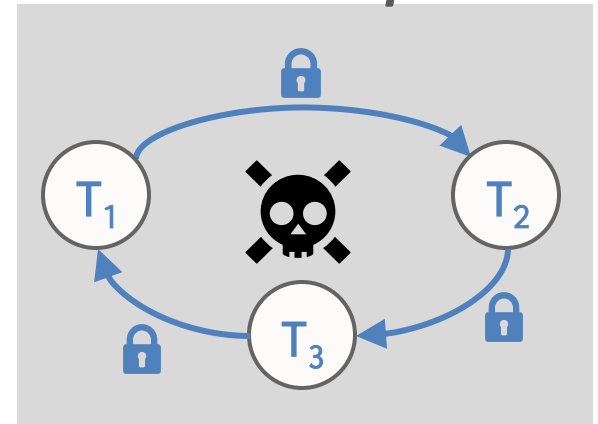
Deadlock Detection

- The DBMS creates a waits-for graph to keep track of what locks each txn is waiting to acquire:
 - Nodes are transactions
 - Edge from T_i to T_j if T_i is waiting for T_j to release a lock.
- The system periodically checks for cycles in waits-for graph and then decides how to break it.

Deadlock Detection



Waits-For Graph



Breaking Deadlocks

- When the DBMS detects a deadlock, it will select a “victim” txn to rollback to break the cycle.
- The victim txn will either restart or abort (more common) depending on how it was invoked.
- There is a trade-off between the frequency of checking for deadlocks and how long txns wait before deadlocks are broken.

Breaking Deadlocks

- Selecting the proper victim depends on a lot of different variables....
 - By age (most/least recent)
 - By progress (least/most queries executed)
 - By the # of items already locked
 - By the # of txns that we have to rollback with it
- We also should consider the # of times a txn has been restarted in the past to prevent starvation.

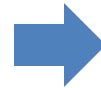
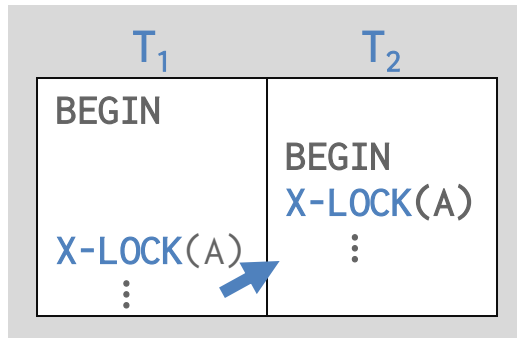
Deadlock Prevention

- When a txn tries to acquire a lock that is held by another txn, the DBMS may pre-emptively kill one of them to prevent a deadlock.
- This approach often does not require an explicit waits-for graph or detection algorithm.
 - At the expense of spurious aborts

Deadlock Prevention

- Assign each txn a timestamp when they start and use them to determine **priorities**.
 - For example, Older Timestamp = Higher Priority (e.g., $T1 > T2$)
- Wait-Die (“**Old Waits for Young**”) **Non-preemptive: holder never aborts**
 - If requesting txn has higher priority than holding txn, then requesting txn waits for holding txn.
 - Otherwise requesting txn aborts.
- Wound-Wait (“**Young Waits for Old**”) **Preemptive**
 - If requesting txn has higher priority than holding txn, then holding txn aborts and releases lock.
 - Otherwise requesting txn waits.

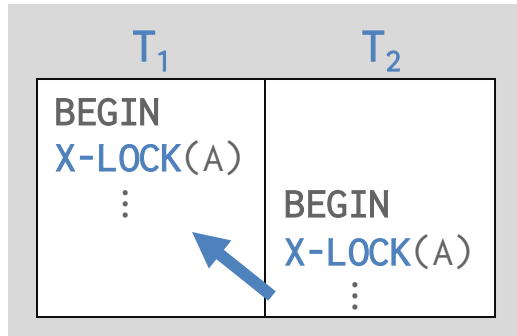
Deadlock Prevention



Wait-Die



Wound-Wait



Wait-Die



Wound-Wait



Deadlock Prevention

- Why do these schemes guarantee no deadlocks?
 - Txns only wait for locks in **one direction**.
- When a txn restarts, what is its (new) priority?
 - Its **original timestamp** to prevent it from getting **starved** for resources.
 - In GoDB – restarts omitted for simplicity

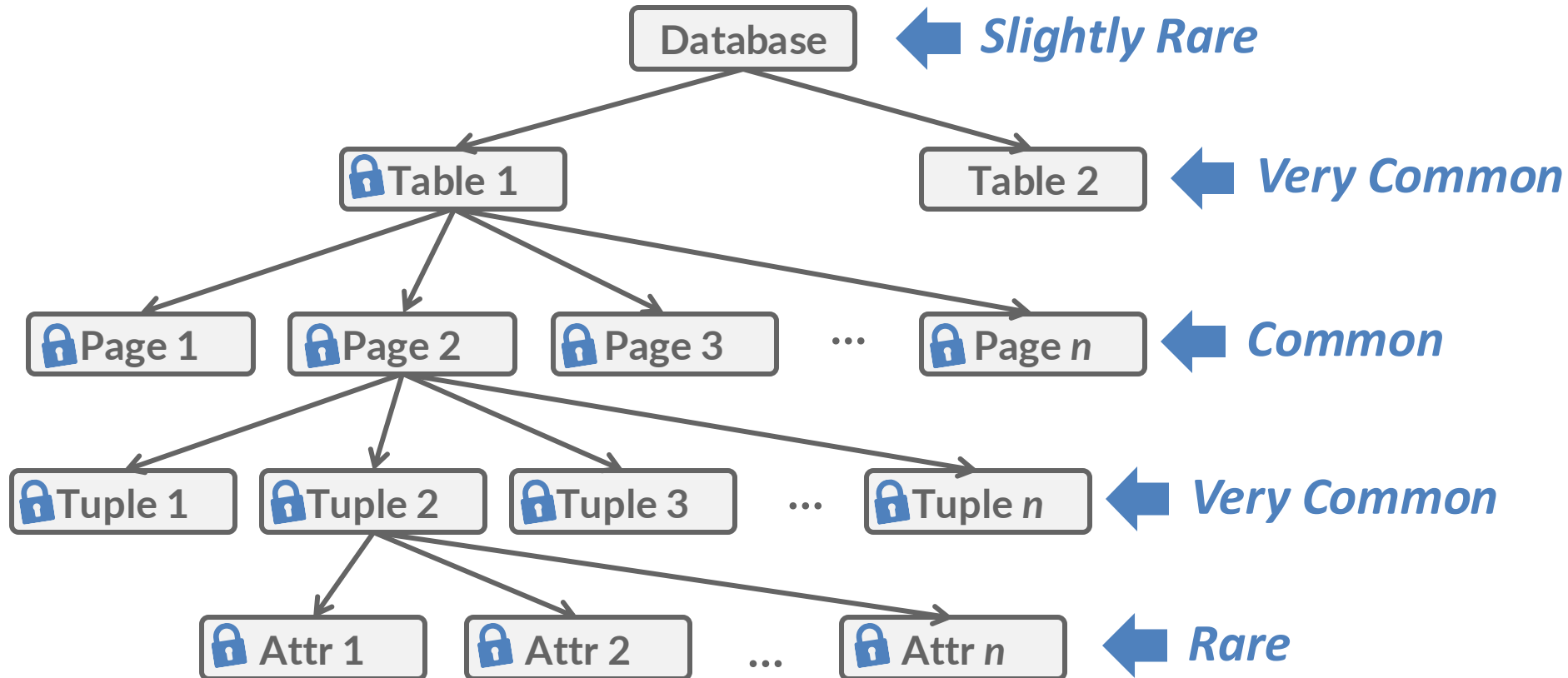
Observation

- All these examples have a one-to-one mapping from database objects to locks.
- If a txn wants to update one billion tuples, then it must acquire one billion locks.
- Acquiring locks is a more expensive operation than acquiring a latch even if that lock is available.
 - Memory overhead, etc.

Lock Granularities

- When a txn wants to acquire a “lock”, the DBMS can decide the granularity (i.e., scope) of that lock.
 - Attribute? Tuple? Page? Table?
- The DBMS should ideally obtain fewest number of locks that a txn needs.
- Trade-off between parallelism versus overhead.
 - Fewer Locks, Larger Granularity vs.
More Locks, Smaller Granularity.

DATABASE LOCK HIERARCHY



Intention Locks

- An intention lock allows a higher-level node to be locked in shared or exclusive mode without having to check all descendent nodes.
- If a node is locked in an intention mode, then some txn is doing explicit locking at a lower level in the tree.

Intention Locks

- Intention-Shared (IS)
 - Indicates explicit locking at lower level with S locks.
 - Intent to get S lock(s) at finer granularity.
- Intention-Exclusive (IX)
 - Indicates explicit locking at lower level with X locks.
 - Intent to get X lock(s) at finer granularity.
- Shared+Intention-Exclusive (SIX)
 - The subtree rooted by that node is locked explicitly in S mode and explicit locking is being done at a lower level with X locks.

Compatibility Matrix

		T_2 Wants				
T_1 Holds		IS	IX	S	SIX	X
	IS	✓	✓	✓	✓	×
	IX	✓	✓	×	×	×
	S	✓	×	✓	×	×
	SIX	✓	×	×	×	×
	X	×	×	×	×	×

Locking Protocol

- Each txn obtains the appropriate lock at highest level of the database hierarchy.
 - Acquire locks top to bottom
- To get S or IS lock on a node, the txn must hold at least IS on parent node.
- To get X, IX, or SIX on a node, must hold at least IX on parent node.

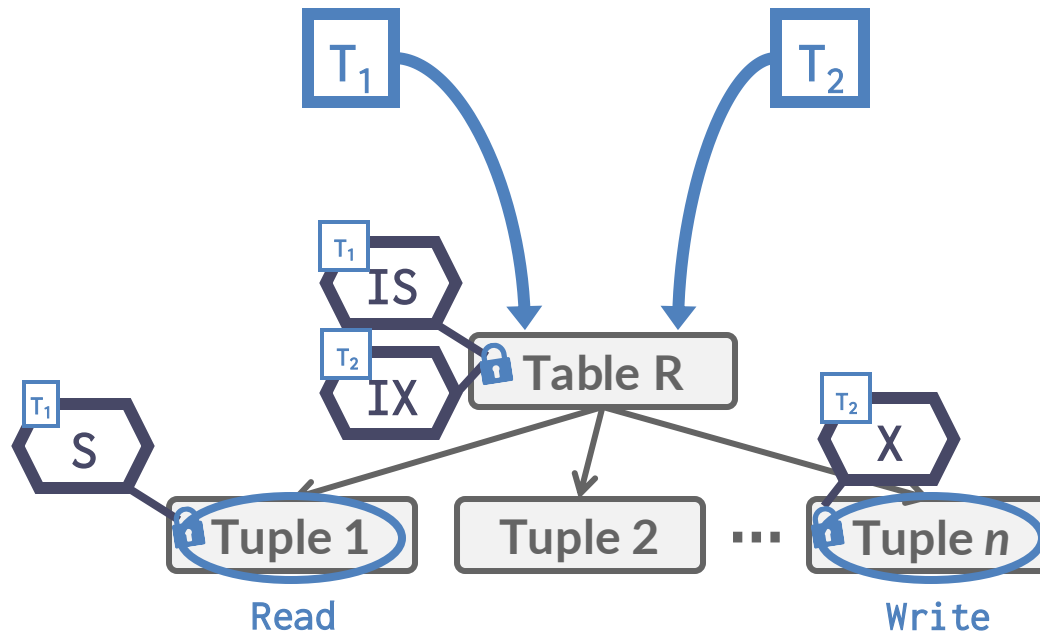
Example

- T1 – Get the balance of MIT's bank account.
- T2 – Increase Tianyu's account balance by \$100.
- What locks should these txns obtain?
 - Explicit Exclusive + Shared locks for leaf nodes of lock tree.
 - Special Intention locks for higher levels.

EXAMPLE: TWO-LEVEL HIERARCHY

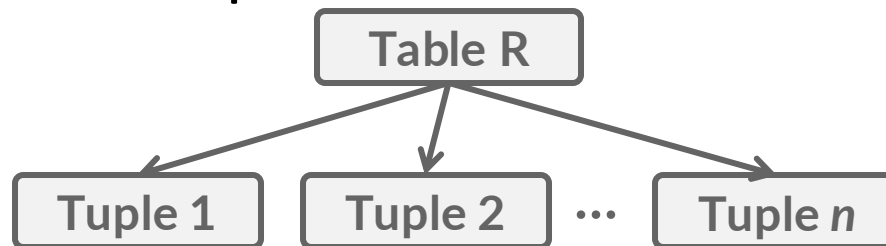
Read MIT's record in **R**.

Update Tianyu's record in **R**.

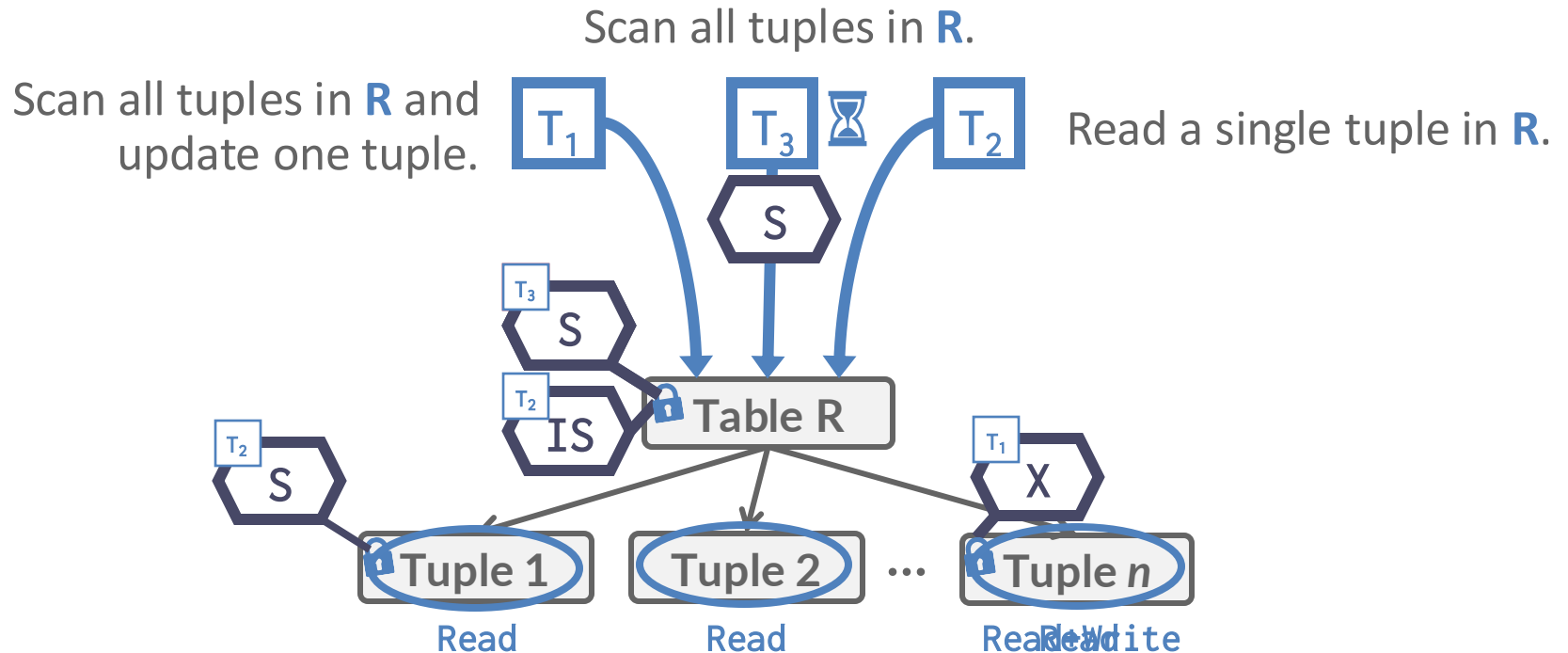


EXAMPLE: THREE TXNS

- Assume three txns execute at same time:
 - T1 – Scan all tuples in R and update one tuple.
 - T2 – Read a single tuple in R.
 - T3 – Scan all tuples in R.



EXAMPLE: THREE TXNS



Lock Escalation

- Some systems automatically switch to coarser-grained locks when a txn acquires too many low-level locks.
 - This reduces the number of requests that the lock manager must process.
- GoDB statically decides lock granularity and push down hints (e.g., forUpdate) at plan time

What are problems with locks?

- Locks are pessimistic.
 - Assume things conflict, prevent conflicts by paying cost
 - If things don't conflict, pessimism is expensive
- Are conflicts common in practice?
 - Yes and no – people actively try to minimize conflicts in the application

Optimistic Concurrency Control (OCC)

- Optimism in concurrency control assumes things do not conflict. Instead, catch it and fix it only when they do
 - Intuitively, fix may be more expensive than prevention
 - but if it happens rarely, it's a net win

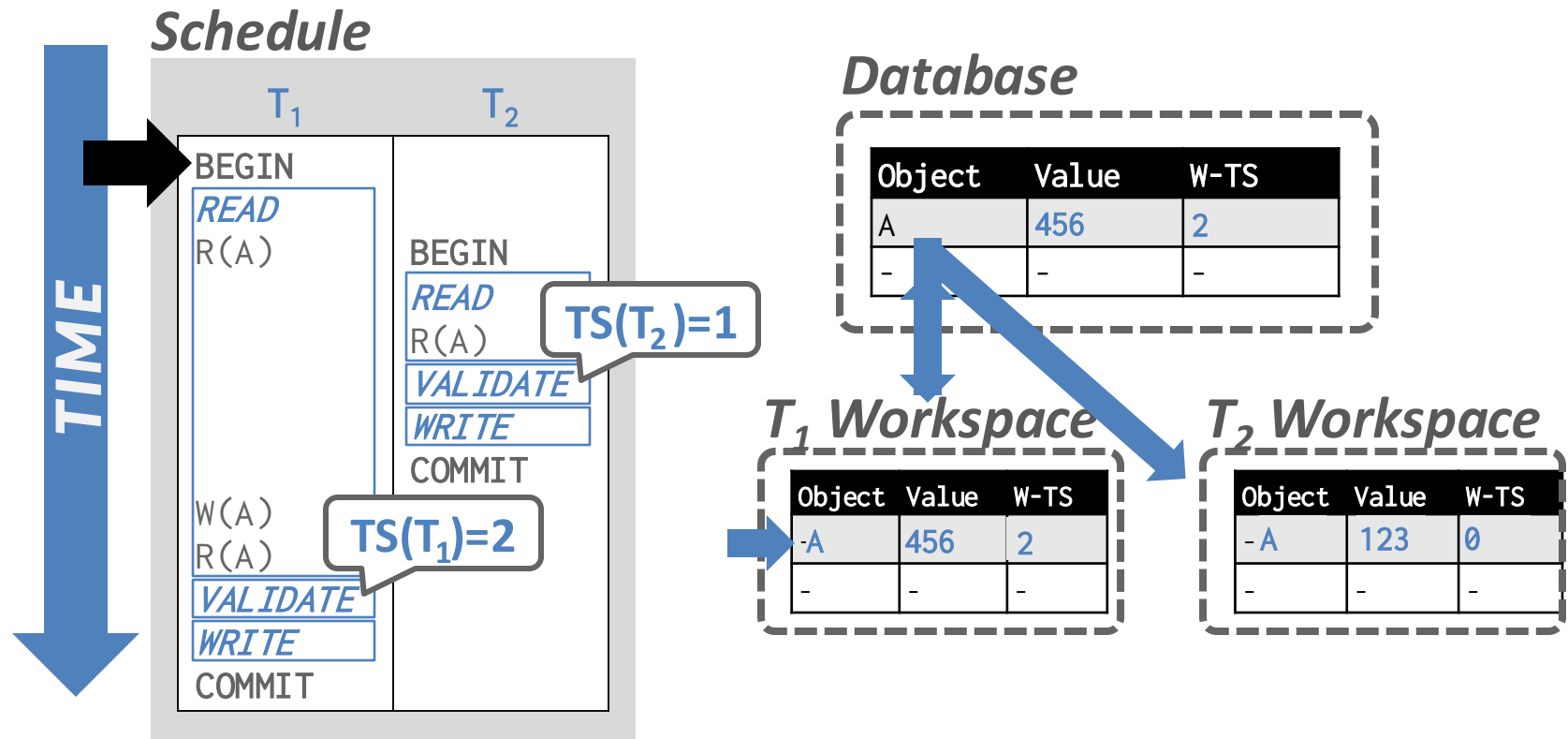
OCC

- The DBMS creates a private workspace for each txn.
 - Any object read is copied into workspace.
 - Modifications are applied to workspace.
- When a txn commits, the DBMS compares workspace write set to see whether it conflicts with other txns.
- If there are no conflicts, the write set is applied and becomes visible.

OCC Phases

- Phase #1: Read
 - Track the read/write sets of txns and store their writes in a private workspace.
 - DBMS copies every tuple that the txn accesses from the shared database to its workspace ensure repeatable reads.
- Phase #2: Validation
 - Assign the txn a unique timestamp (TS) and then check whether it conflicts with other txns.
- Phase #3: Write
 - If validation succeeds, set the write timestamp (W-TS) to all modified objects in private workspace and install them into the global database. Otherwise abort txn.

OCC Phases



OCC: Read Phase

- Track the read/write sets of txns and store their writes in a private workspace.
- The DBMS copies every tuple that the txn accesses from the shared database to its workspace ensure repeatable reads.
 - Details are hand-wavy for now

OCC: Validation Phase

- When txn T_i invokes COMMIT, the DBMS checks if it conflicts with other txns.
 - Original OCC algorithm uses serial validation.
 - Parallel validation requires each txn check read/write sets of other txns trying to validate at the same time.
- DBMS needs to guarantee only serializable schedules are permitted.
 - Approach #1: Backward Validation
 - Approach #2: Forward Validation

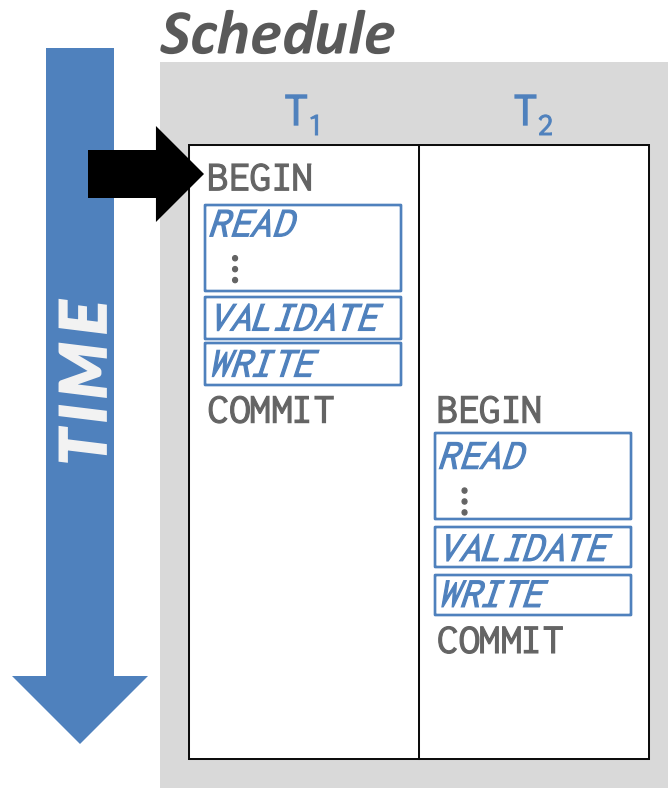
OCC: Validation Phase

- Forward Validation: Check whether the committing txn intersects its read/write sets with any active txns that have not yet committed.
- Backward Validation: Check whether the committing txn intersects its read/write sets with those of any txns that have already committed.

OCC: Forward Validation

- The DBMS assigns the txn a unique timestamp at the beginning of the validation phase.
- Check the timestamp ordering of the committing txn with all other active txns.
 - An active txn is one that has not committed yet.
- If $TS(T1) < TS(T2)$, then one of the following three conditions must hold...

CASE #1

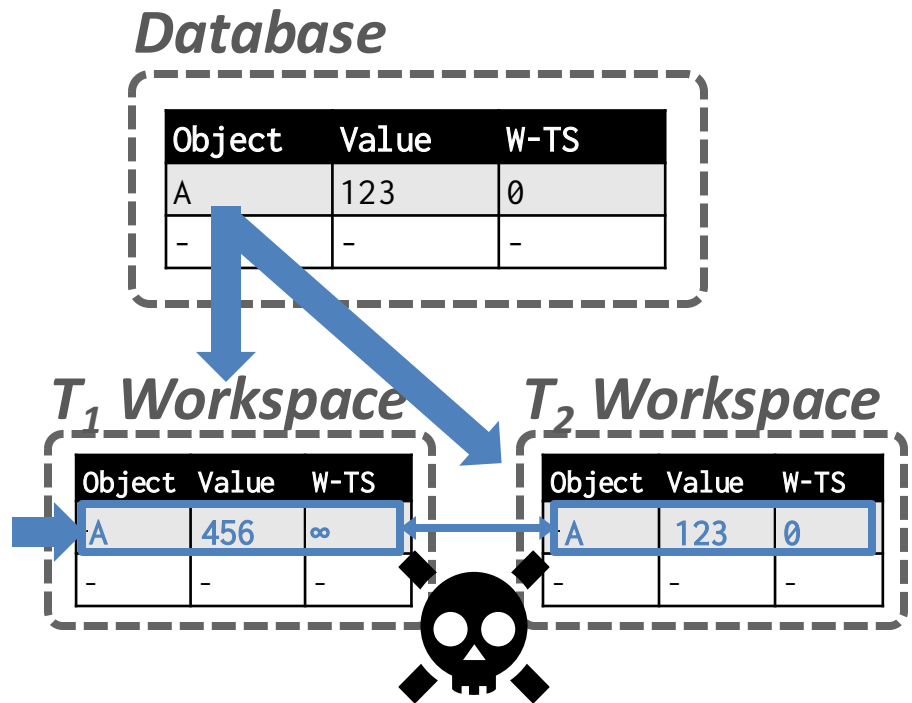
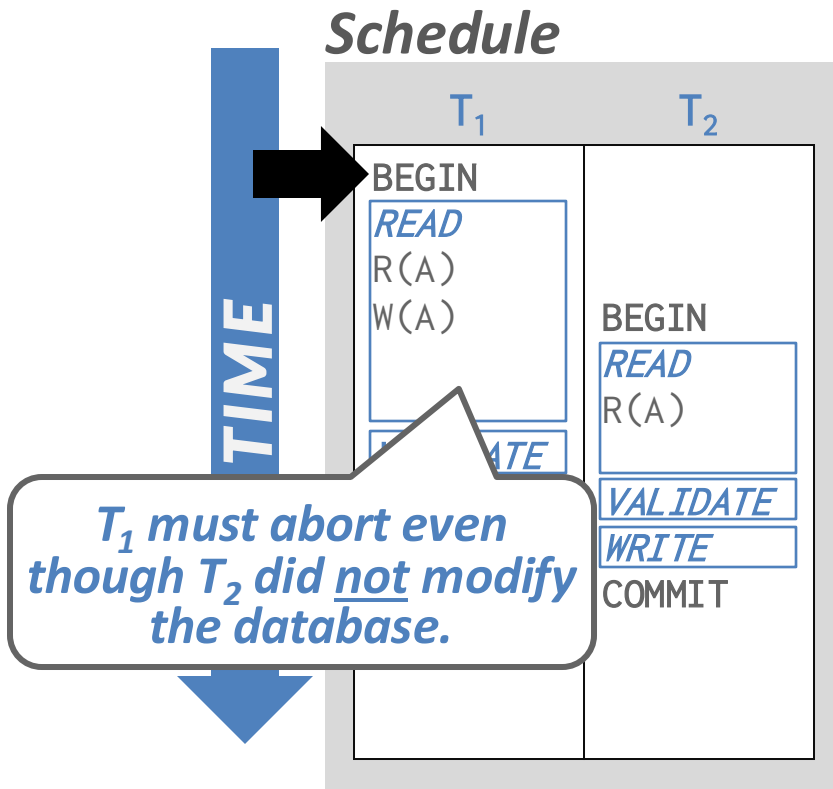


- Example: T1 wants to commit.
- If ($T_1 < T_2$), check if T1 completes its Write phase before T2 begins its Read phase.
- No conflict as all T1's actions happen before T2's.
 - This just means that there is serial ordering.

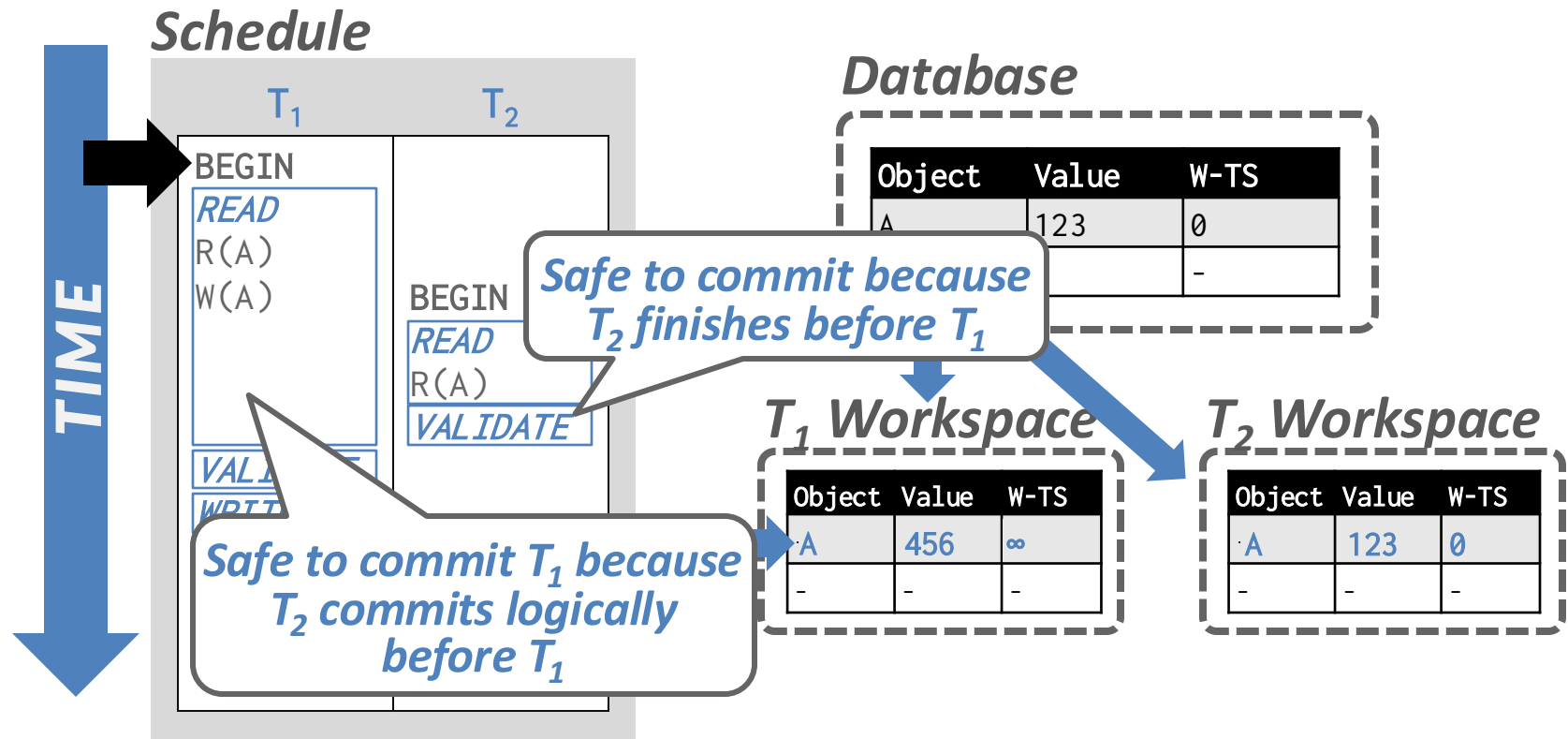
CASE #2

- Example: T1 wants to commit.
- If ($T1 < T2$), check if T1 completes its Write phase before T2 starts its Write phase and T1 does not modify to any object read by T2.
 - The intersection of T1's WriteSet with T2's ReadSet is empty:
 $\text{WriteSet}(T1) \cap \text{ReadSet}(T2) = \emptyset$

CASE #2



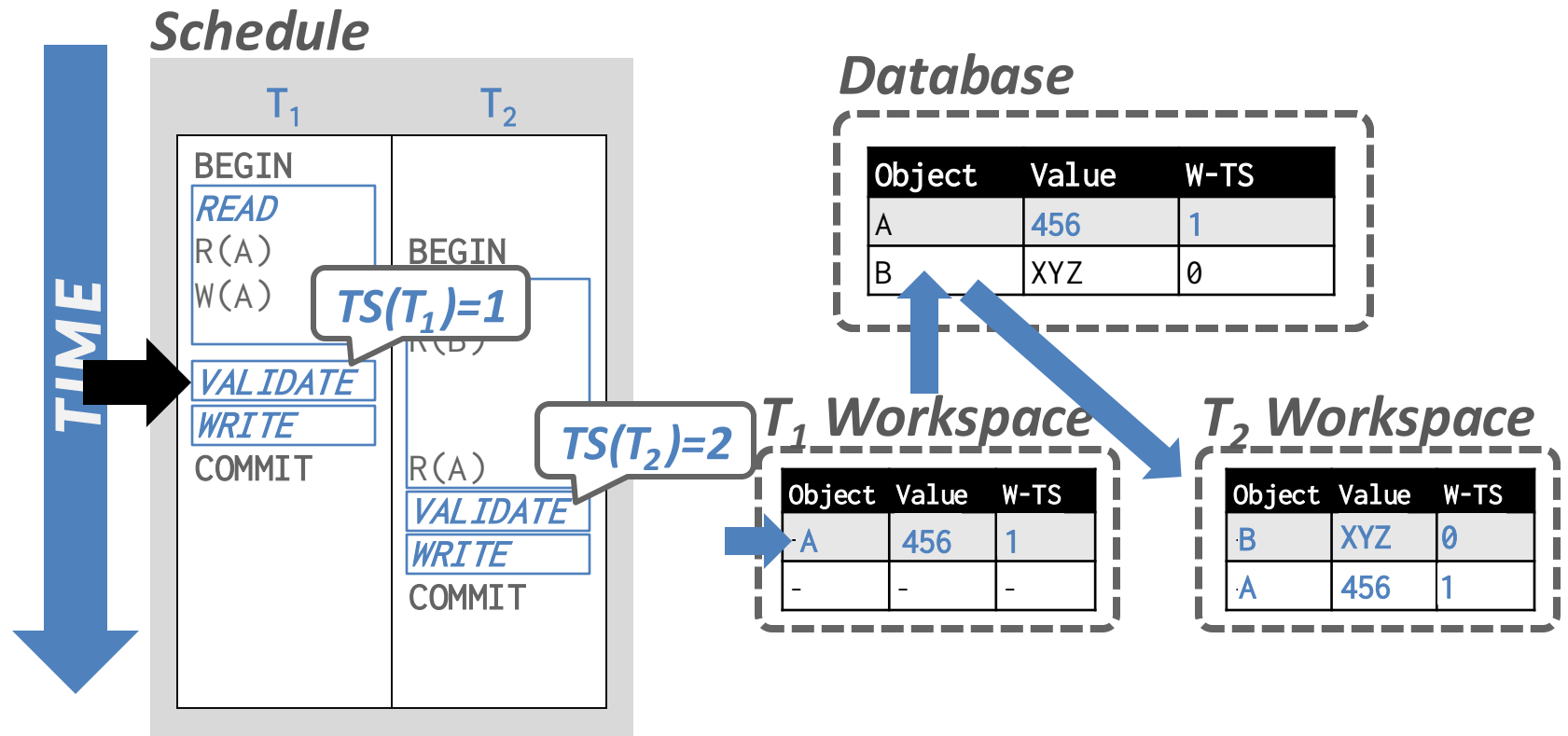
CASE #2



CASE #3

- Example: T1 wants to commit.
- If ($T1 < T2$), check if T1 completes its Read phase before T2 completes its Read phase and T1 does not modify any object either read or written by T2:
 - $\text{WriteSet}(T1) \cap \text{ReadSet}(T2) = \emptyset$
 - $\text{WriteSet}(T1) \cap \text{WriteSet}(T2) = \emptyset$

CASE #3



OCC: Write Phase

- Propagate changes in the txn's write set to database to make them visible to other txns.
- Serial Commits:
 - Use a global latch to limit a single txn to be in the Validation/Write phases at a time.
- Parallel Commits:
 - Use fine-grained write latches to support parallel Validation/Write phases.

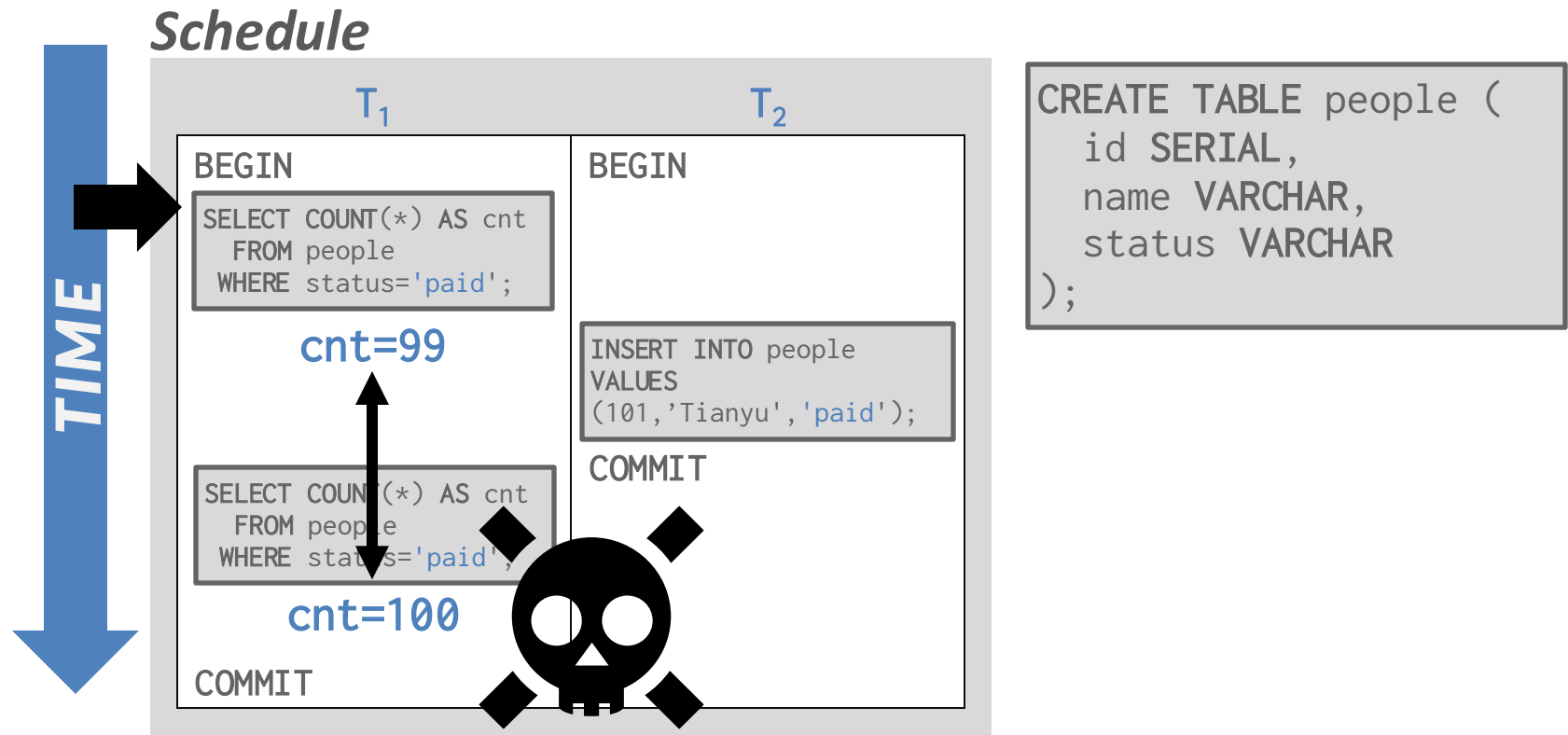
OCC: Summary

- OCC works well when the number of conflicts is low:
 - Many txns are read-only (ideally all).
 - Txns access disjoint subsets of data.
- But OCC has its own problems:
 - High overhead for copying data locally.
 - Validation/Write phase bottlenecks.
 - Aborts are more wasteful than in 2PL because they only occur after a txn has already executed.

OBSERVATION

- We have only dealt with transactions that read and update **existing** objects in the database.
- But now if txns perform insertions, updates, and deletions, we have new problems...

THE PHANTOM PROBLEM



OOPS?

- How did this happen?
 - Because T1 **locked only existing records** and not ones that other txns are adding to the database!
- Conflict serializability on reads and writes of individual items guarantees serializability **only if the database's set of objects is fixed.**
- This is known as a [phantom read](#).
 - A txn scans a range more than once and another txn inserts/removes tuples that fall within that range in between the scans.

Solutions

- Approach #1: Lock Everything!  *Less Common*
 - Entire table or every page.
- Approach #2: Re-Execute Scans  *Rare*
 - Run queries again at commit to see whether they produce a different result to identify missed changes.
- Approach #3: Predicate Locking  *Rare*
 - Logically determine the overlap of predicates before queries start running.
- Approach #4: Index Locking  *Common*
 - Use keys in indexes to protect ranges.

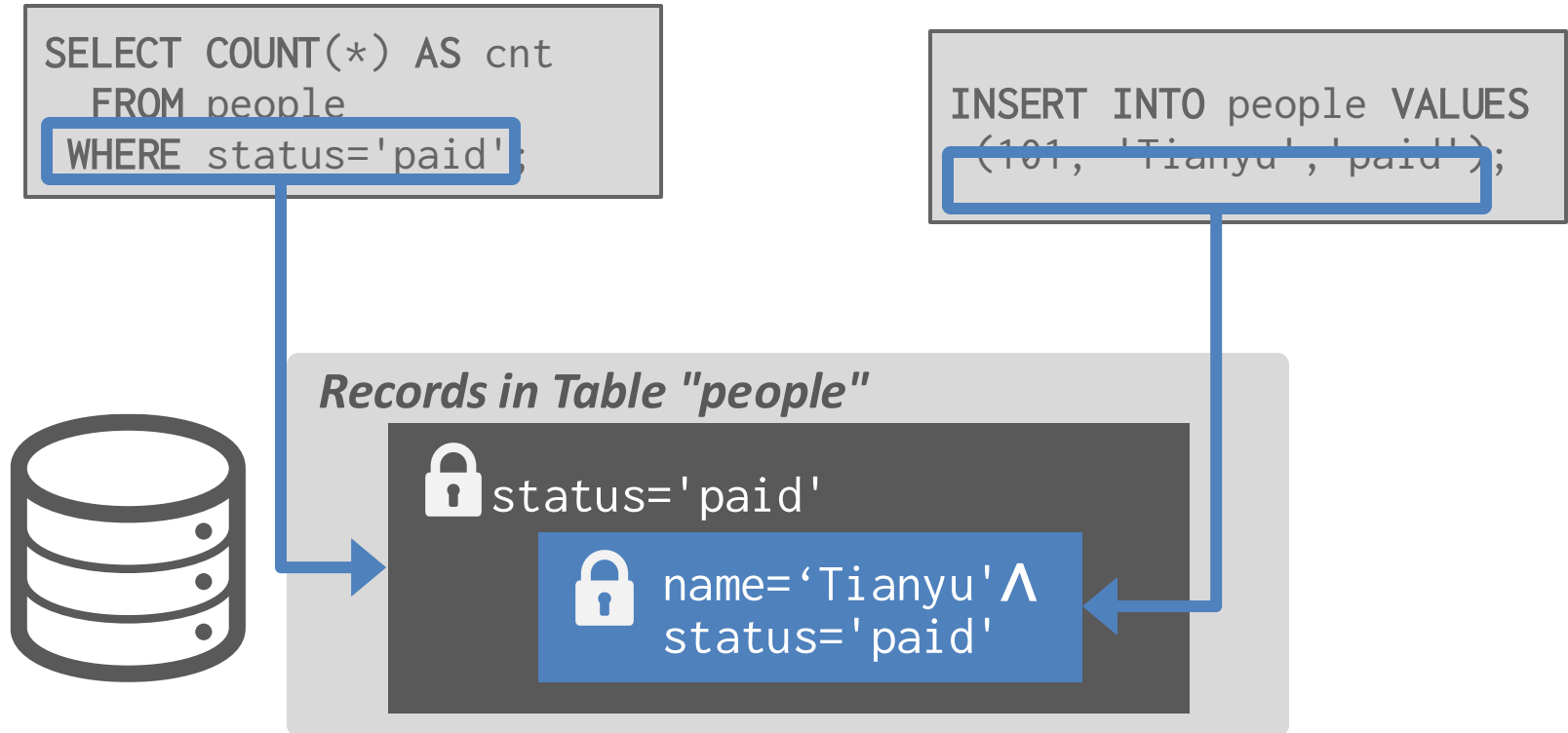
Re-execute Scans

- The DBMS tracks the WHERE clause for all queries that the txn executes.
 - Retain the scan set for every range query in a txn.
- Upon commit, re-execute just the scan portion of each query and check whether it generates the same result.
 - Example: Run the scan for an UPDATE query but do not modify matching tuples.

Predicate Locking

- Proposed locking scheme from System R.
 - Acquire a Shared lock on the predicate in a WHERE clause of a SELECT query.
 - Acquire an Exclusive lock on the predicate in a WHERE clause of any UPDATE, INSERT, or DELETE query.
 - Some systems approximate it via [precision locking](#).
- Others do it via index locking...

Predicate Locking



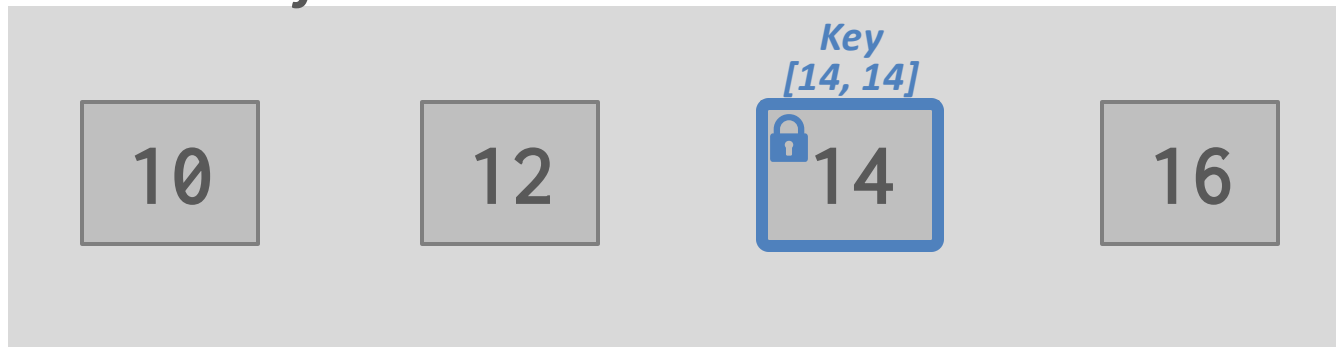
Index Locking

- Special case of predicate locking that acquires locks on key ranges in indexes.
 - If there is an index on the status attribute then the txn locks index page containing the data with status='paid'.
 - If there are no records with status='paid', the txn locks the index page where such a data entry would be, if it existed.
- Different approaches:
 - Key-Value Locks
 - Gap Locks
 - Key-Range Locks
 - Hierarchical Locking

Key-Value Locks

- Locks that cover a single key-value in an index.
- Need “virtual keys” for non-existent values.

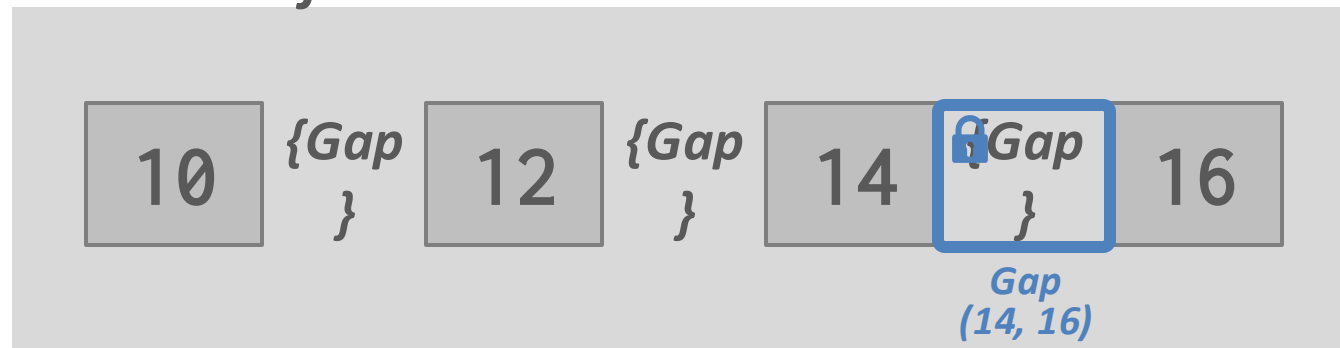
B+Tree Leaf Node



Gap Locks

- Each txn acquires a key-value lock on the single key that it wants to access. Then get a gap lock on the next key gap.

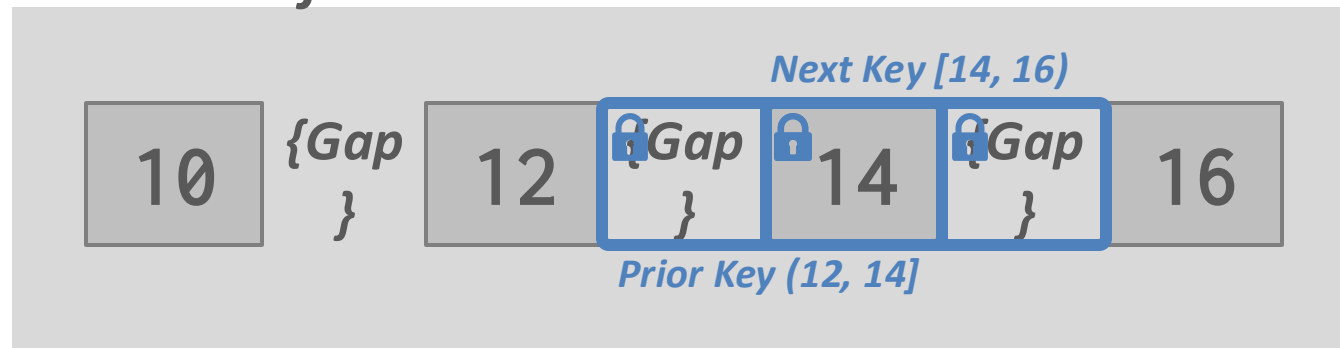
B+Tree Leaf Node



Key-Range Locks

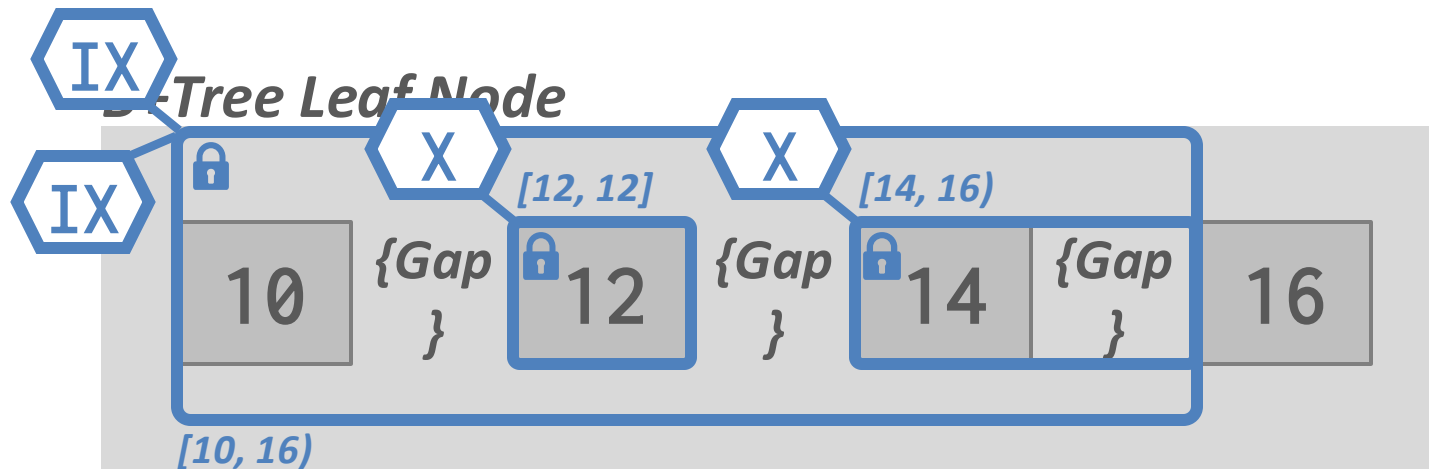
- Locks that cover a key value and the gap to the next key value in a single index.
 - Need “virtual keys” for artificial values (infinity)

B+Tree Leaf Node



Hierarchical Locking

- Allow for a txn to hold wider key-range locks with different locking modes.
 - Reduces the number of visits to lock manager.



Isolation Levels

- Serializability is useful because it allows programmers to ignore concurrency issues.
- But enforcing it is hard and may allow too little concurrency and limit performance.
- The DBMS may want to use a weaker level of consistency to improve parallelism opportunities.

Isolation Levels

- Controls the extent that a txn is exposed to the actions of other concurrent txns.
- Provides for greater concurrency at the cost of exposing txns to anomalies:
 - Dirty Reads
 - Unrepeatable Reads
 - Lost Updates
 - Phantom Reads

Isolation Levels



	<i>Dirty Read</i>	<i>Unrepeatable Read</i>	<i>Lost Updates</i>	<i>Phantom</i>
SERIALIZABLE	No	No	No	No
REPEATABLE READ	No	No	No	Maybe
READ COMMITTED	No	Maybe	Maybe	Maybe
READ UNCOMMITTED	Maybe	Maybe	Maybe	Maybe

Isolation Levels

- **SERIALIZABLE**: Strong Strict 2PL with phantom protection (e.g., index locks).
- **REPEATABLE READS**: Same as above, but without phantom protection.  *GoDB*
- **READ COMMITTED**: Same as above, but S locks are released immediately.
- **READ UNCOMMITTED**: Same as above but allows dirty reads (no S locks).

Isolation Levels

- The application can set a txn's isolation level before it executes any queries in that txn.
- Not all DBMS support all isolation levels in all execution scenarios
 - There are many more isolation levels, and despite standards, people only roughly agree on what they mean...
- The default depends on implementation...
 - But rarely actually **SERIALIZABLE**

Takeaways

- Concurrency Control is hard
 - And we have only scratched the surface
- Next two lectures:
 - Modern solution: MVCC
 - Recovery with ARIES
- Enjoy Spring Break!